Final Project: Perform Vulnerability Analysis and Penetration Testing

Lab Report

SecureBank — Vulnerability Analysis, Penetration Testing & Data Protection
Date: July 14, 2026

Overview

As an ethical hacker hired by SecureBank, a mid-sized financial institution, this project identifies a vulnerability that could expose the bank's systems to cyber threats and builds a penetration testing plan around it. Part 1 uses IBM X-Force Exchange to identify a recent CVE, investigates it further with three progressively refined Google Dorking commands, and works through six scenario-based decisions that shape the penetration testing plan. Part 2 demonstrates a practical data-protection workflow, using OpenSSL in a Linux Cloud IDE to generate an AES key, encrypt a sensitive user-details file, and verify the encryption is fully reversible.

Environment	IBM X-Force Exchange, Google Search, Linux Cloud IDE (Theia)
Tools Used	IBM X-Force Exchange, Google Dork operators (site:, filetype:), OpenSSL (rand, enc, -aes256-cbc, -pbkdf2), nano, cat, ls
Tasks Completed	4 of 4 (CVE identification, Google Dorking investigation, penetration testing plan, data encryption and decryption)

Task 1: Identify a Vulnerability Using IBM X-Force Exchange

Objective: *Use IBM X-Force Exchange to find a recent vulnerability that has been assigned a CVE number, and capture a screenshot in which the CVE number is clearly visible.*

1.1 IBM X-Force Exchange Vulnerability Report

I searched IBM X-Force Exchange and identified CVE-2026-12581, a Session Fixation vulnerability (CWE-384) in Digiwin's EasyFlow .NET business process management software. The report shows a CVSS 3.1 base score of 7.5 (High), an attack vector of Network with high complexity, and confirms that an unauthenticated attacker who replaces a victim's session ID can gain that user's privileges once they log in. The report also lists the remedy: updating to version 8.1.5 or later.

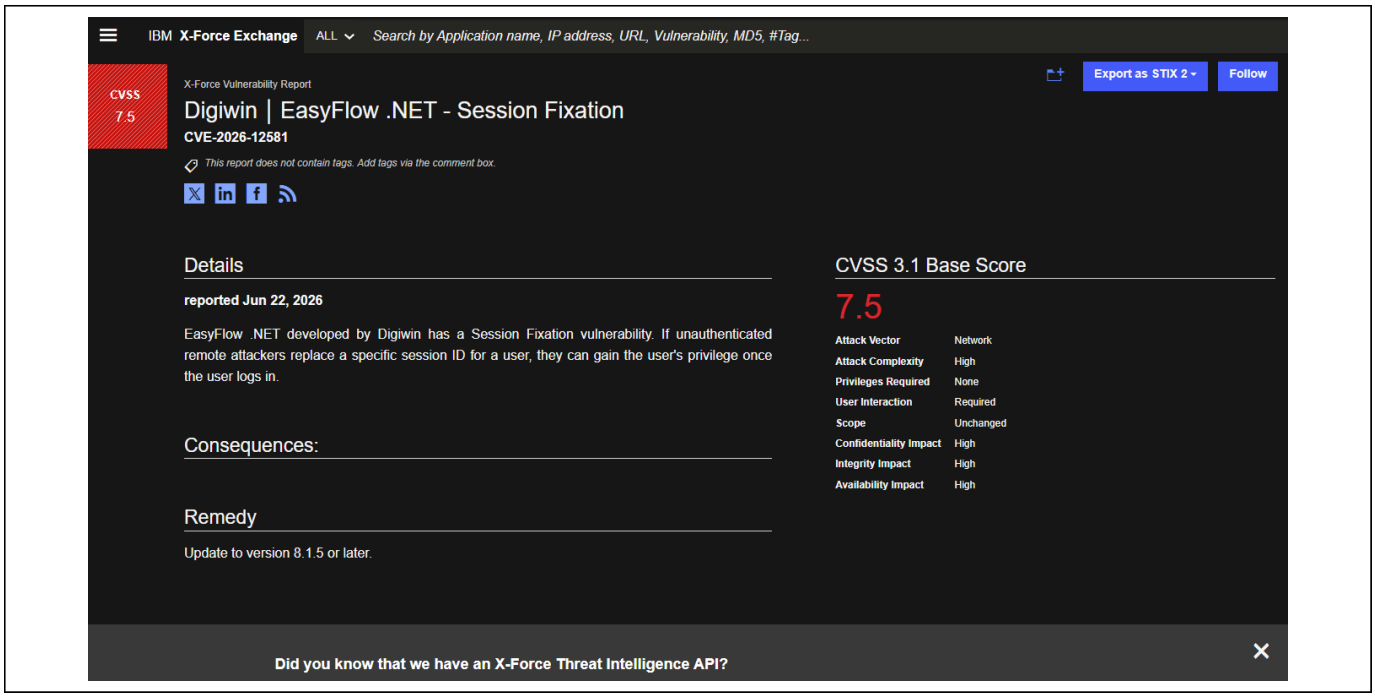

Figure 1. IBM X-Force Exchange report for CVE-2026-12581, showing the CVSS 3.1 base score, vulnerability details, and remedy.

Task 2: Investigate the Selected Vulnerability Using Google Dorking Commands

Objective: *Use three progressively refined Google Dorking commands — site:, site:gov, and filetype:pdf — to investigate CVE-2026-12581 and locate authoritative sources discussing it.*

2.1 Step 1: site: Search for the CVE Number

I searched Google using the dork site: CVE-2026-12581 to find external sources discussing the same vulnerability. The results returned the official CVE.org record, the NIST NVD detail page, and third-party vulnerability trackers (SentinelOne, IONIX, Aqua Security, and VulDB), all referencing CVE-2026-12581 and confirming it as the EasyFlow .NET session fixation issue.

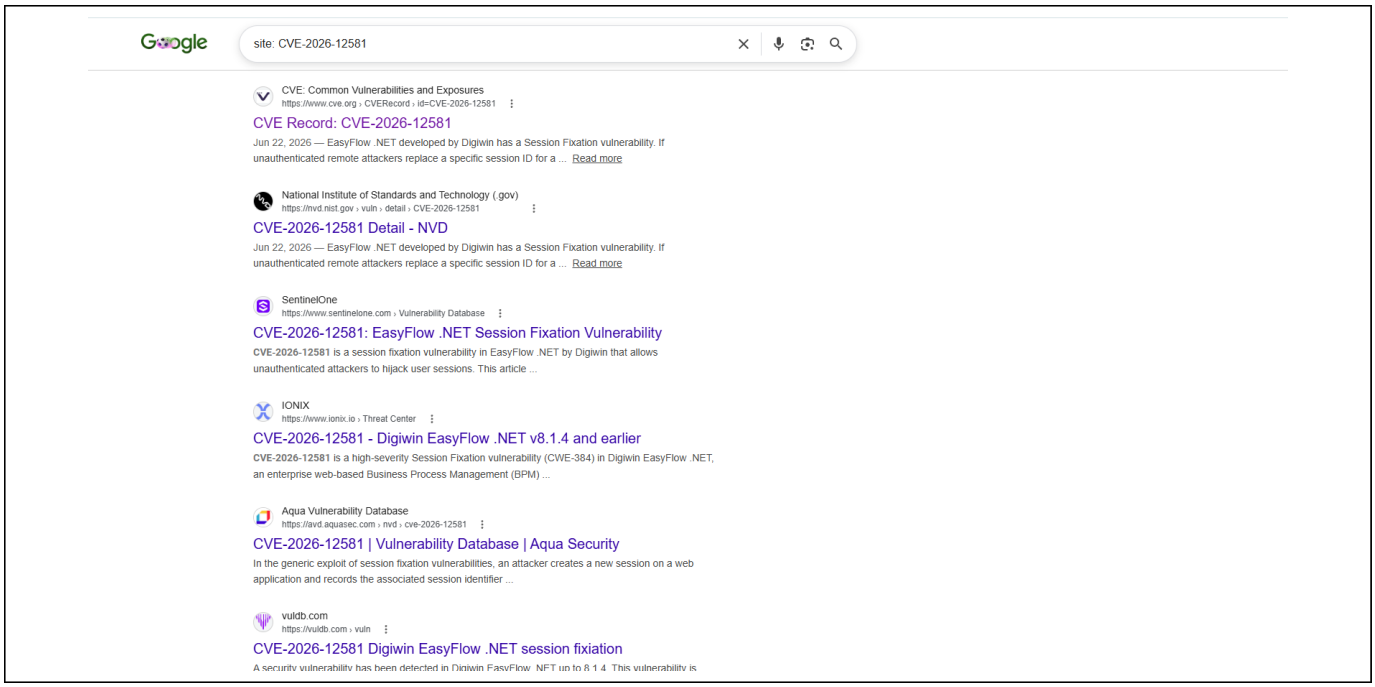

Figure 2. Google search results for the site: CVE-2026-12581 dork, showing CVE.org, NIST NVD, and third-party vulnerability database entries.

2.2 Step 2: Refine the site: Command to .gov Websites

To narrow the results to authoritative government sources, I refined the query to site:gov CVE-2026-12581. This returned results exclusively from NIST NVD and CISA, including the CVE-2026-12581 detail page and a CISA weekly vulnerability bulletin referencing the same CVE, confirming the vulnerability through official .gov channels.

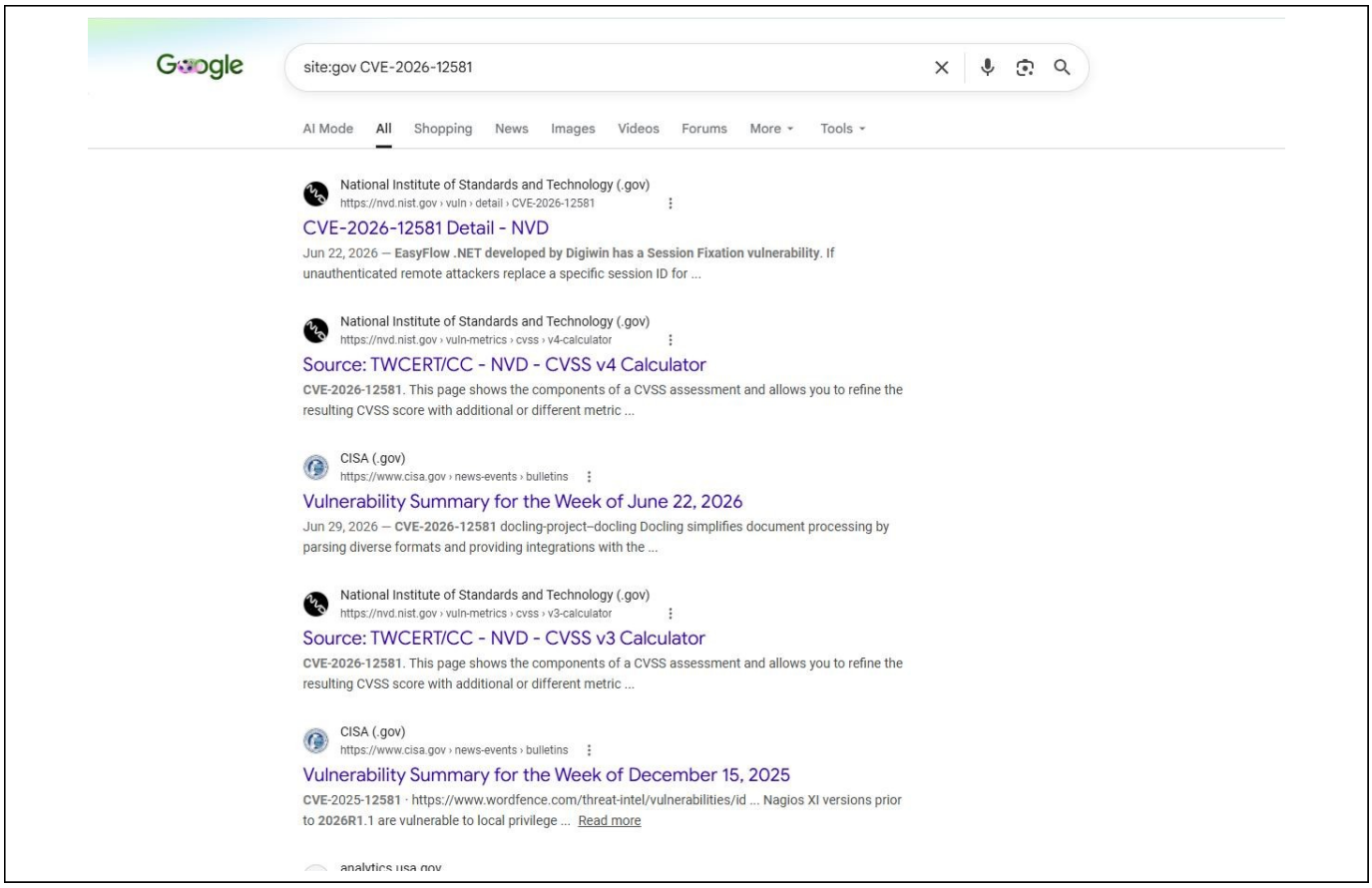

Figure 3. Refined Google Dork site:gov CVE-2026-12581, returning results limited to NIST and CISA .gov domains.

2.3 Step 3: Use the filetype: Command to Find PDF Reports

I further refined the search with site:gov CVE-2026-12581 filetype:pdf to look for PDF-formatted vulnerability documentation on .gov sites. The search returned several PDF results from .gov domains; while the CVE number appeared in the query, none of the returned PDF titles or snippets explicitly referenced CVE-2026-12581 in the visible preview text.

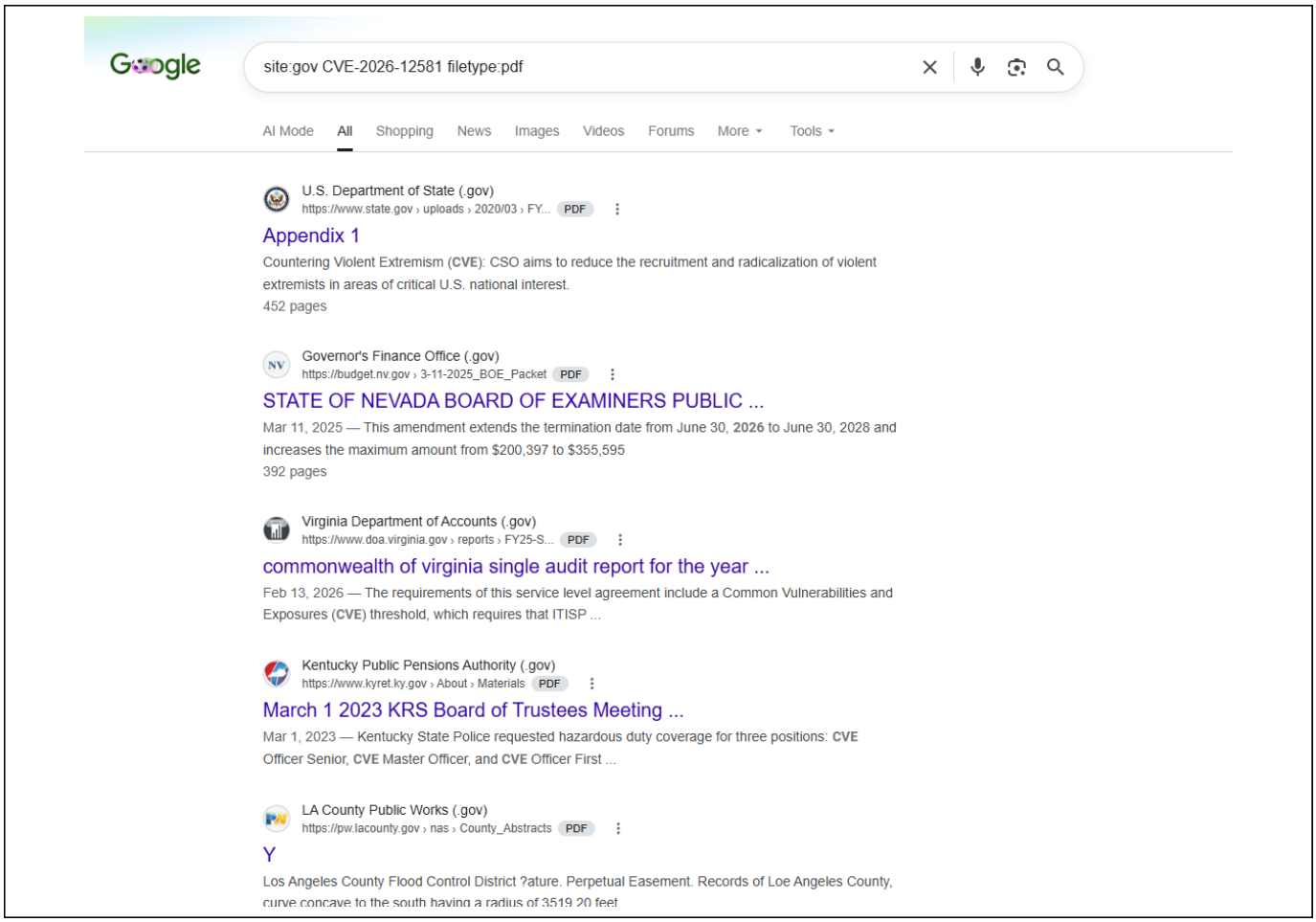

Figure 4. Google Dork site:gov CVE-2026-12581 filetype:pdf, showing PDF results restricted to .gov domains.

Task 3: Create a Penetration Testing Plan

Objective: *Work through six scenario-based steps for SecureBank's penetration test, selecting the best option for each scenario and providing a justification for the choice.*

3.1 Step 1: Defining the Test Scope

Selected option: A — Include all systems, networks, and applications.

Justification: SecureBank's infrastructure includes external web applications, internal databases, and a mix of on-premises and cloud-based services. Because these environments are highly interconnected, limiting the scope to only external or only internal systems risks missing lateral-movement paths between in-scope and out-of-scope assets, and a comprehensive scope also aligns with regulatory expectations for demonstrating full due diligence.

3.2 Step 2: Prioritizing the Objective Based on IT Team Concerns

Selected option: B — Test incident response capabilities.

Justification: Given the IT team's concerns about recent phishing attacks, verifying how quickly and effectively the team can detect and contain a phishing-driven compromise is more immediately valuable than a broad vulnerability sweep, since it directly addresses the risk the IT team raised.

3.3 Step 3: Minimizing Disruption to Business Operations

Selected option: B — Notify staff that the test will be conducted after business hours.

Justification: As a bank, SecureBank cannot risk interfering with live financial transactions during business hours. Notifying staff and scheduling the test after hours reduces the risk of disrupting customer-facing services and ensures the bank's team is prepared to respond if the testing triggers any alerts.

3.4 Step 4: Gathering Information in the Discovery Phase

Selected option: C — Use a combination of automated and manual techniques.

Justification: With limited information about SecureBank's network topology, automated tools such as Nmap and Nessus can efficiently map the attack surface and flag known vulnerabilities at scale, while manual techniques are needed to uncover context-specific weaknesses that automated scanners alone would miss.

3.5 Step 5: Exploiting the Identified Vulnerabilities

Selected option: B — Exploit the highest severity vulnerabilities first.

Justification: Among the outdated web server, weak passwords, and unpatched database identified, prioritizing the highest-severity vulnerabilities first demonstrates the most business-critical risks early in the engagement, giving SecureBank's leadership the clearest picture of where the greatest exposure lies.

3.6 Step 6: Presenting the Findings

Selected option: C — Create a detailed technical report and an executive summary.

Justification: The audience includes both technical staff and executive leadership. A detailed technical report gives IT and security staff the specifics needed for remediation, while an executive summary translates the findings into business language that communicates risk and impact to leadership.

Part 2 — Task 4: Data Encryption and Decryption with OpenSSL

Objective: *Create a file containing sensitive user data, generate an AES key, encrypt the file, confirm the ciphertext is unreadable, and then decrypt it to verify the original data is recovered intact.*

4.1 Create the userdetails File

I used nano to create a userdetails file containing a sample user record (username, password, email, and phone number), then confirmed its contents with cat userdetails.

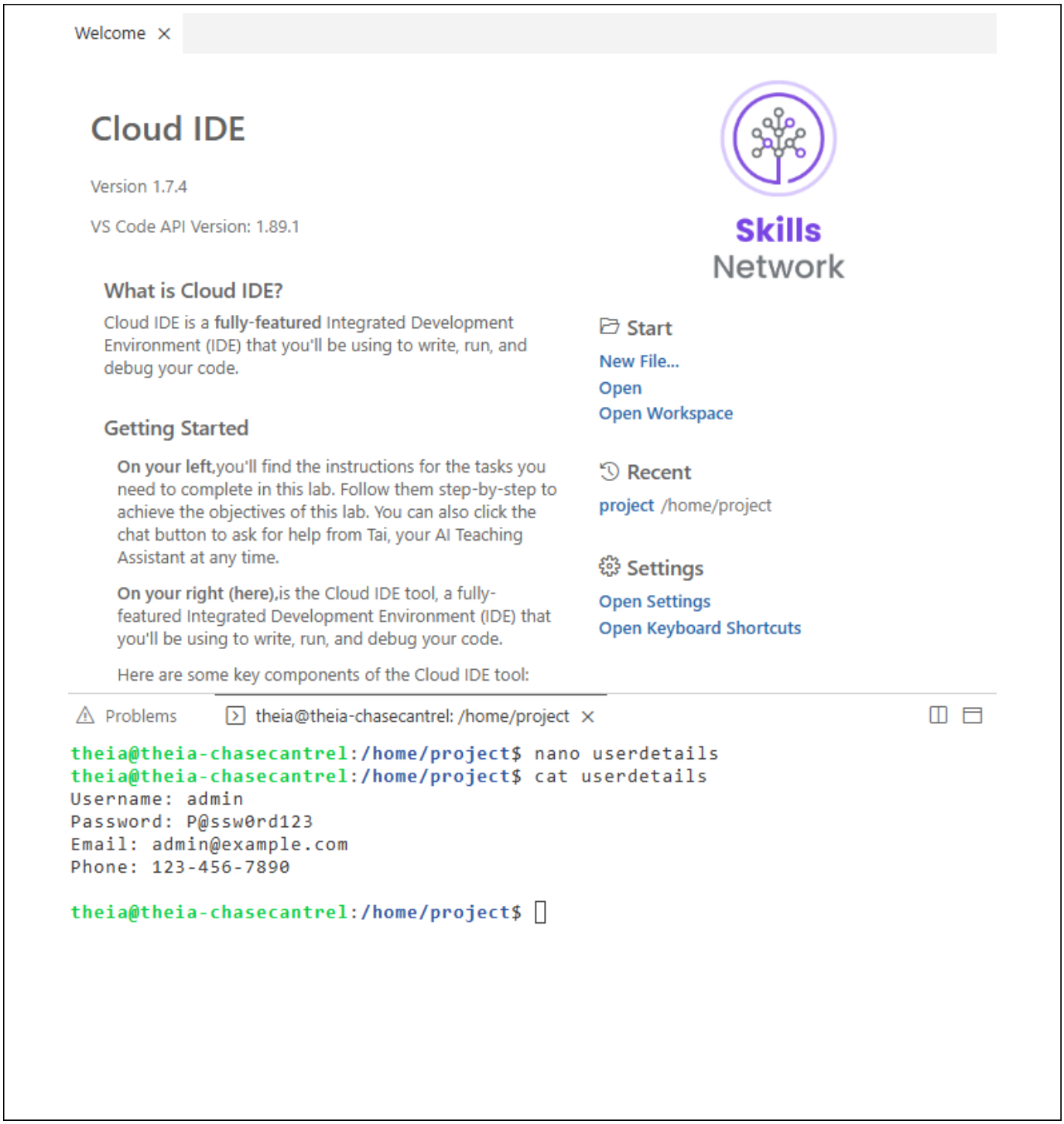

Figure 5. Creating userdetails with nano and confirming its contents with cat.

4.2 Generate an AES Key

I generated a random 32-byte key with openssl rand -base64 32 > passkey, then used ls -l to confirm both userdetails and the newly created passkey file were present in the directory.

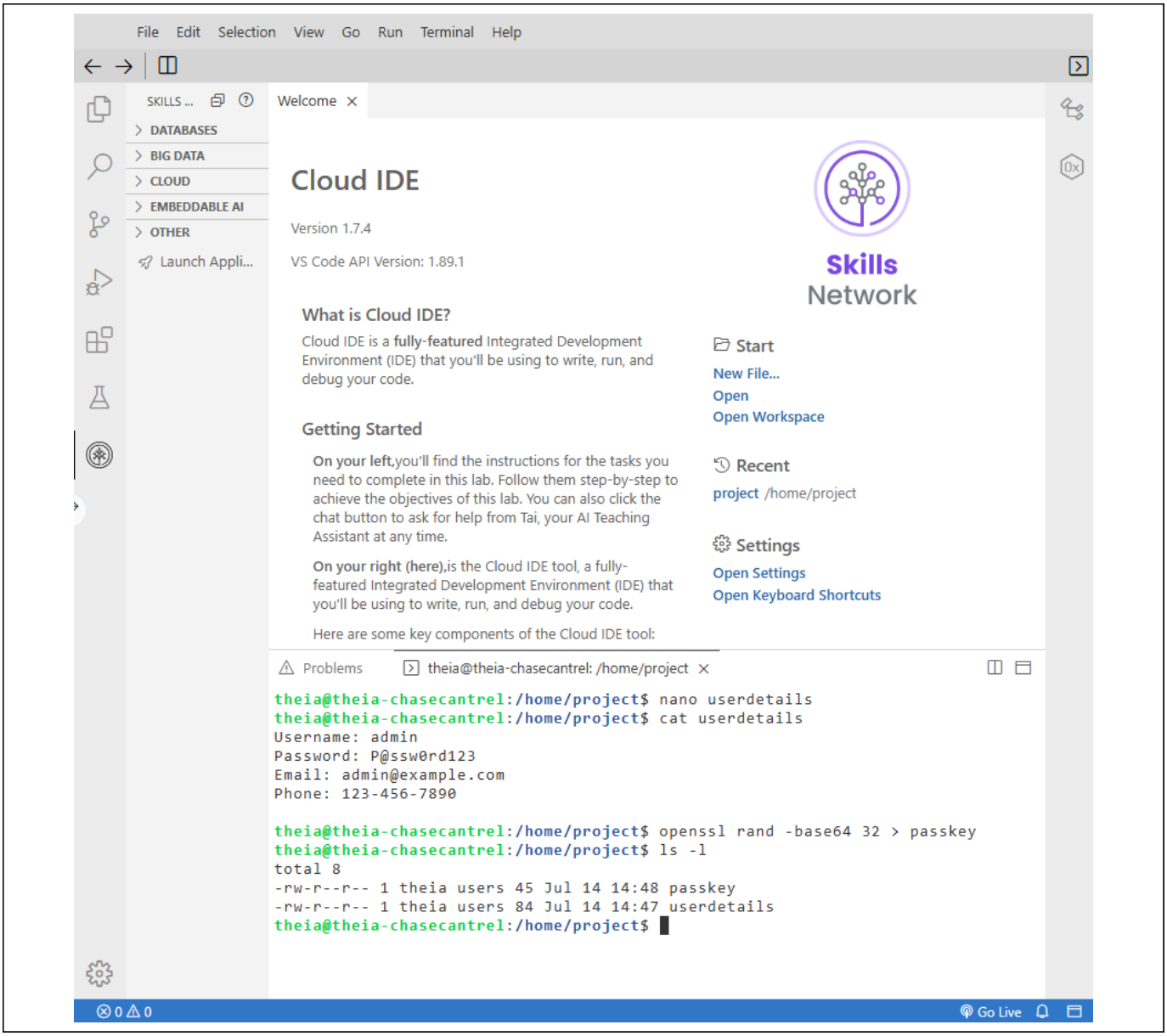

Figure 6. Generating the AES key with openssl rand -base64 32 and verifying the passkey file was created.

4.3 Encrypt the userdetails File

I encrypted the file with openssl enc -aes-256-cbc -salt -pbkdf2 -in userdetails -out userdetails_encrypt -pass file:passkey. Running ls -l afterward confirmed userdetails_encrypt was created.

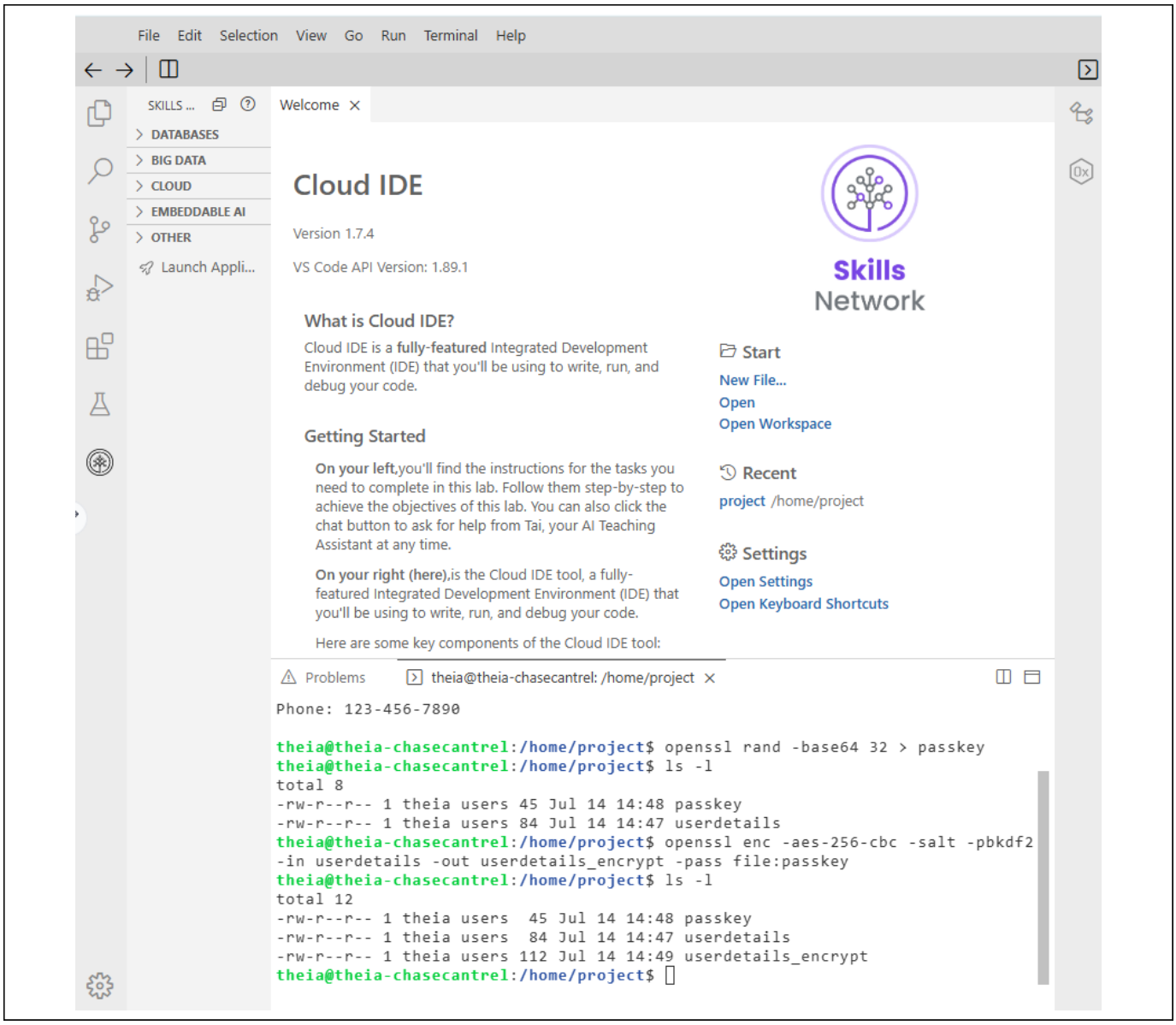

Figure 7. Encrypting userdetails with AES-256-CBC and PBKDF2, then verifying the new encrypted file with ls -l.

4.4 Confirm the Encrypted Output Is Unreadable

I ran cat userdetails_encrypt to view the encrypted file directly. The command's output shows only salted, unreadable binary data — beginning with the "Salted__" header followed by non-printable characters — with no legible plaintext anywhere in the result. This confirms that the original username, password, email, and phone number are no longer exposed in the stored file.

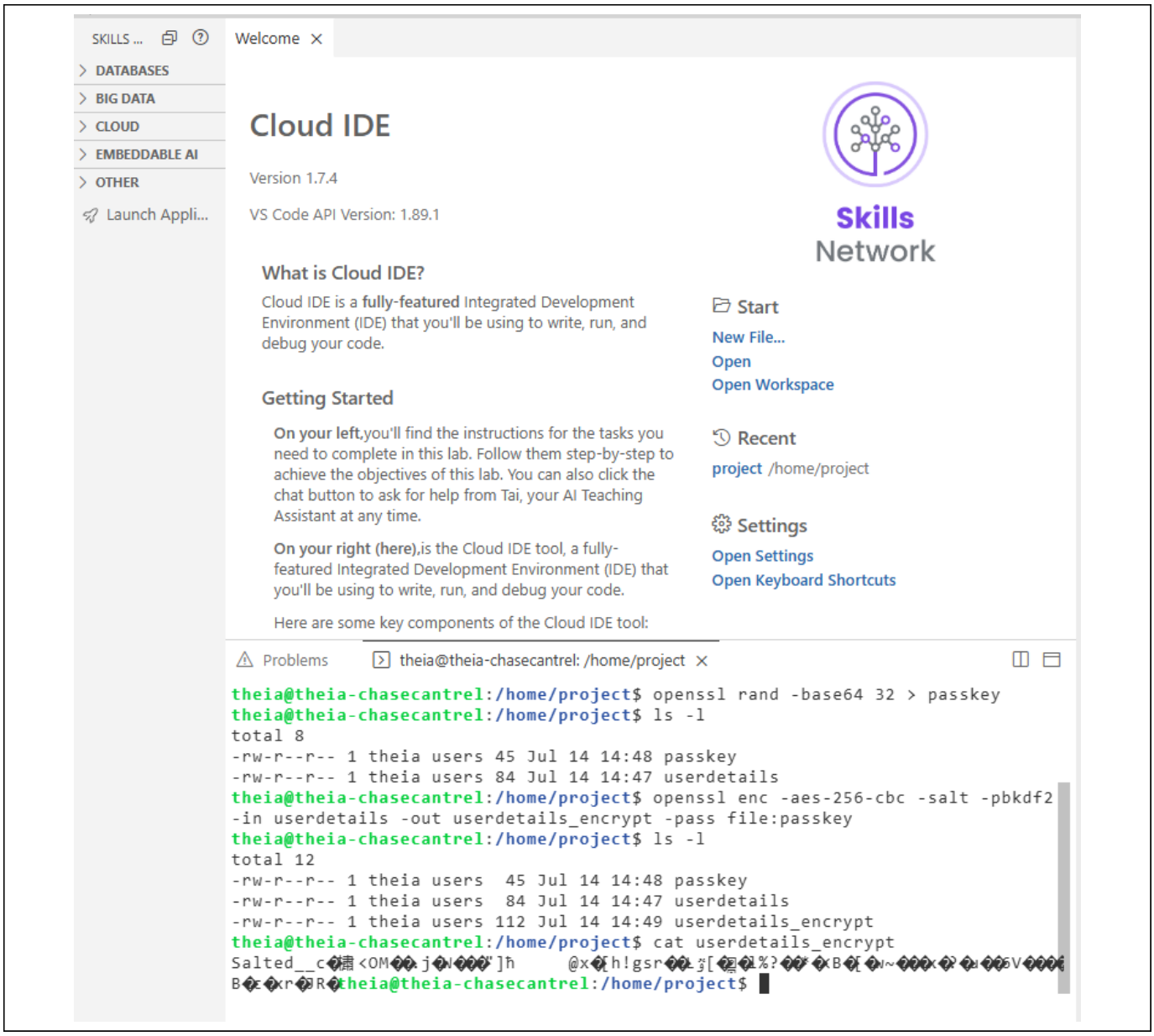

Figure 8. Directory listing after encryption, confirming userdetails_encrypt was created.

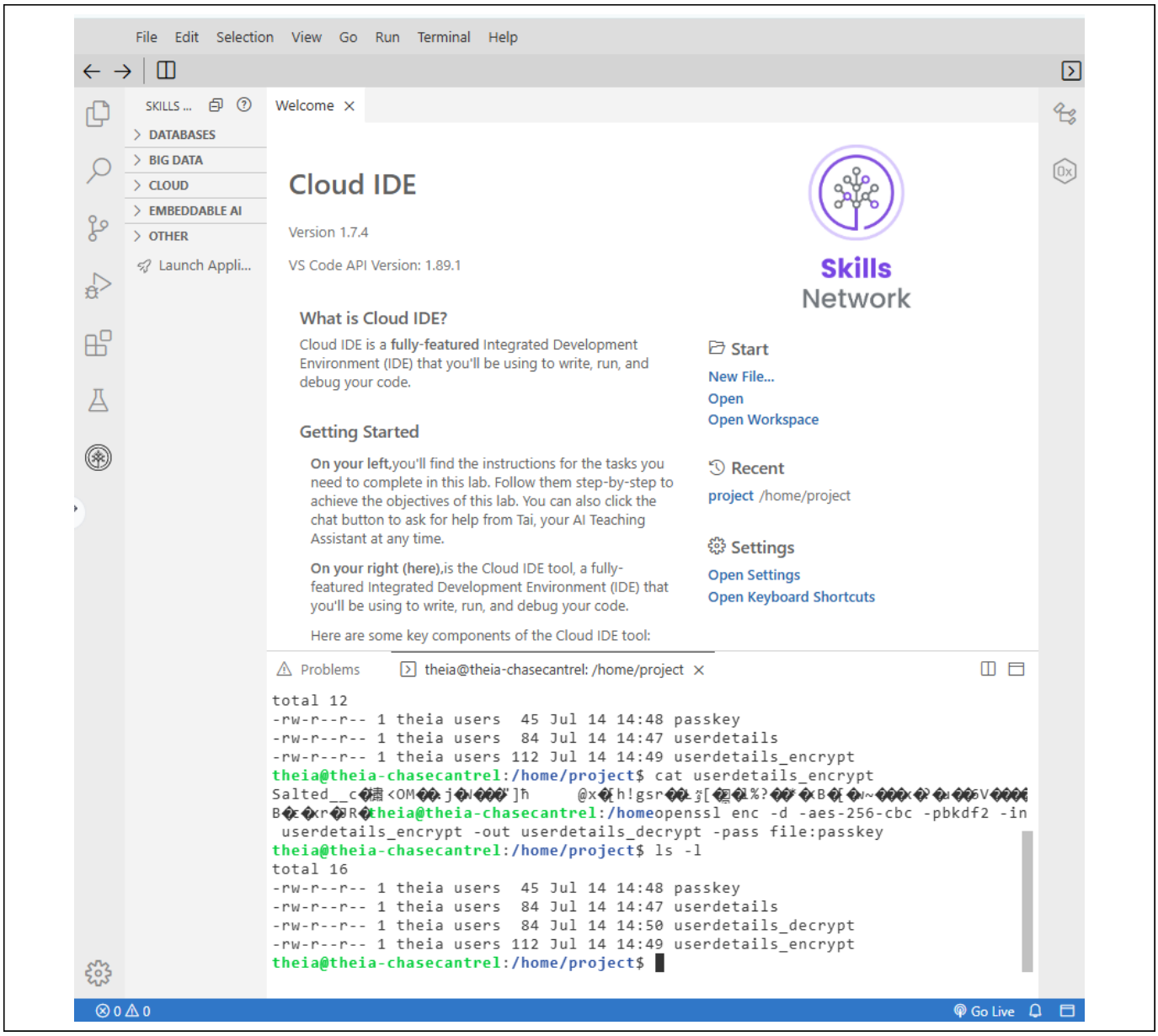

Figure 9. Command cat userdetails_encrypt and its output: unreadable, salted binary ciphertext with no visible plaintext.

4.5 Decrypt and Verify

I reversed the process with openssl enc -d -aes-256-cbc -pbkdf2 -in userdetails_encrypt -out userdetails_decrypt -pass file:passkey, then ran ls -l to confirm userdetails_decrypt was created alongside the original files. I then ran cat userdetails_decrypt, and the command's output returned the recovered plaintext record, matching the original userdetails file exactly — the same username, password, email, and phone number — confirming that the encryption was fully reversible with the correct AES key and that no data was lost or altered in the encrypt/decrypt cycle.

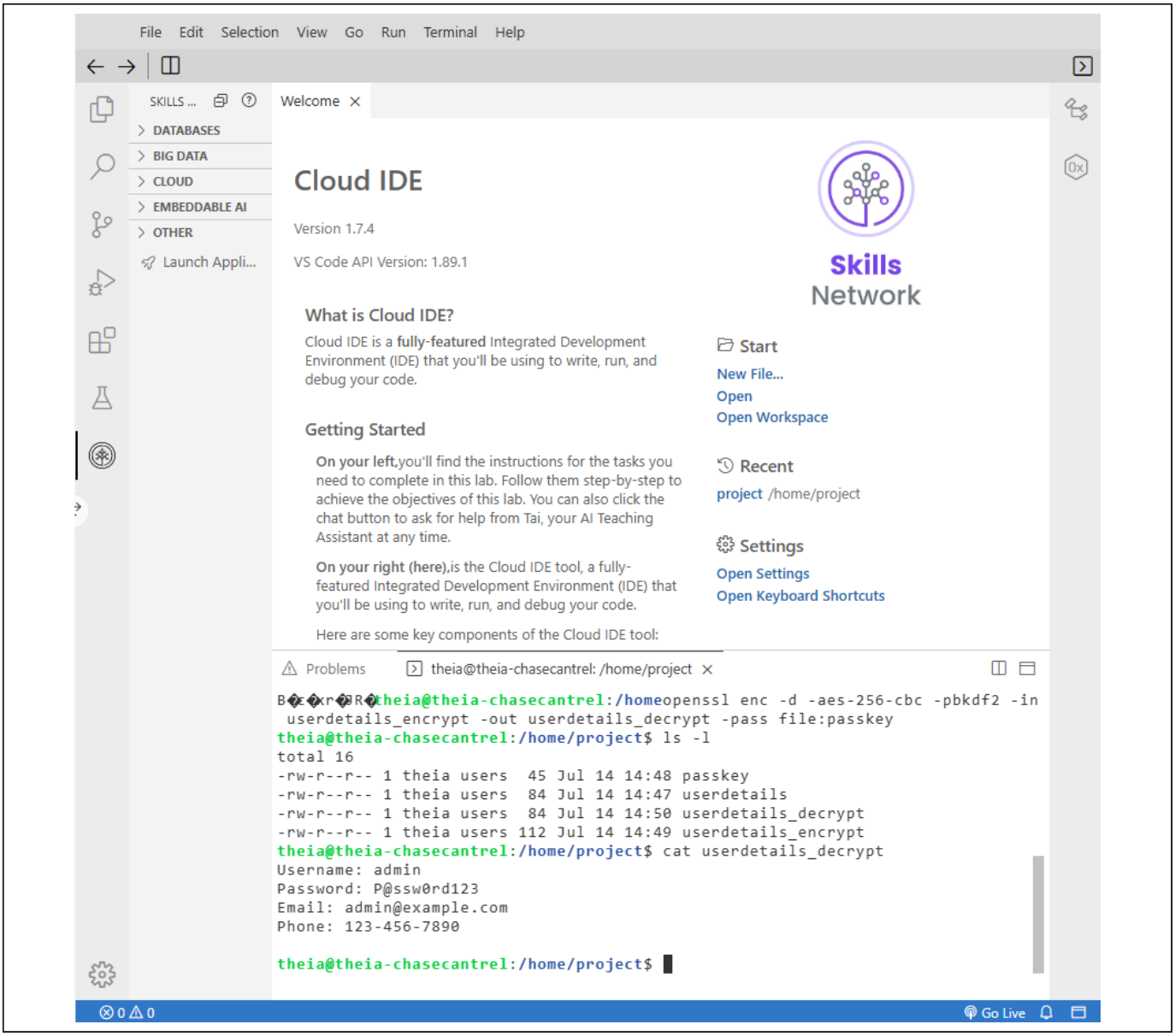

Figure 10. Command cat userdetails_decrypt and its output, showing the fully recovered plaintext that matches the original userdetails record.

Conclusion

All tasks for this project were completed. I identified CVE-2026-12581 on IBM X-Force Exchange and investigated it further using three progressively refined Google Dork queries — a broad site: search, a site:gov filter, and a filetype:pdf search of government sources. I then built a six-step penetration testing plan for SecureBank, selecting and justifying an option at each step: a full-scope test, prioritizing incident response testing, scheduling after hours, combining automated and manual discovery techniques, exploiting the highest-severity vulnerabilities first, and delivering both a technical report and an executive summary. Finally, I demonstrated a complete data-protection workflow in the Cloud IDE: creating a sensitive user-details file, generating an AES key with OpenSSL, encrypting the file to unreadable ciphertext, and decrypting it back to plaintext that matches the original record exactly. Together, these tasks cover vulnerability research, penetration test planning, and practical encryption — three core pillars of SecureBank's security assessment.